Reducing Bank-Synchronization Cycles of a Zero-Skipping FPGA CNN Core by Offline Channel Permutation

(Author names)
(Affiliation)

(e-mail)

Abstract—A zero-skipping convolution core whose input banks run in lockstep waits, in every window, for the bank with the most nonzero activations. In our integer CNN core for a Zynq XC7Z020 FPGA, the bank of a tap is its index mod 4 in a (ky, kx, channel) window, so where the channel count is a multiple of 4 the channel order decides which channels share a bank. We fix this order offline on calibration images and fold it into the adjacent layers; the bitstream and the integer output do not change, and a per-window cycle model shows from the layer shapes alone where this cannot pay. At four values per cycle, two relabelings of a 32-channel denoiser cut the core cycles measured on one image by 7.02% and 7.60% (transcribed from run notes), as a model pushed before the relabeled networks were exported predicted to the cycle once reported DMA stalls were removed. On test images not used for calibration the counted saving was 5.1–10.9% and 5.6–11.2% per image, and under a protocol pushed before the recipe was fitted on nine of the networks the pooled W = 4 count fell on all ten networks studied. Tap-level reorderings, which this host also permits, were not evaluated systematically; a preliminary count (Section VII) suggests that they save more.

Keywords—zero skipping, activation sparsity, load balancing, channel permutation, FPGA, CNN accelerator, cycle model

I. Introduction

A core that skips zero activations saves multiplications [1], [2], but units that advance together make each window cost as much as the busiest one: Cnvlutin measured this lane-synchronization stall [1], and SCNN idle cycles at barriers between processing elements [3]. Known remedies for activation imbalance change the network or add hardware: S2TA and a patent bound the nonzeros per block or bank by discarding activations [4], [5], and Sense and Crane rebalance at run time [6], [7]. Sense notes that input maps "cannot be trained offline to balance IFM workload" [6]. Offline, function-preserving reorderings exist, but they balance weights [8], [9], [10].

In our integer core four tap banks run in lockstep, and in 32- and 64-channel layers all nine taps of a channel land in the bank given by its index modulo 4. The exported channel order thus sets which channels share a bank, and a reordering can be folded into the adjacent layers [8], [11], so the function stays exact while nonzeros move between banks.

We ask when such a relabeling can save cycles at all, and how much of the bank-imbalance cost one fixed relabeling, chosen on calibration images, removes on other images; as activations change with the input, a static assignment can balance the banks only on average. The contributions are (i) a necessary condition, from the layer shapes alone, for a relabeling to change a layer's cycles (Section II); (ii) a board measurement with identical output, predicted exactly by a model pushed before the relabeled networks were exported (Section V-A); and (iii) a ten-network study under a protocol pushed before the fits, with baselines (Sections V-B to V-D). Tap-level layouts are left open (Section VII).

II. Core, Cost Model and When a Relabeling Can Pay

Core. The host builds each layer's windows (im2col), sends them with the layer's weights and requantizes between layers; the core computes every convolution with INT8 weights, 7-bit unsigned activations and INT32 accumulation. Its 64 multipliers form P = 16 output lanes times T = 4 tap banks, and its window port accepts W values per cycle (W = 1 and 4 have run on the board, W = 2 only in RTL simulation). In sparse mode, used throughout, each bank processes only its nonzero taps, and each group of P outputs takes as many cycles as the fullest bank has nonzero taps.

Cost of a window. For a window with K taps and COUT outputs, and m the largest number of nonzero taps in one bank, the core spends

$$C = \max(\lceil K/W \rceil + 1, \lceil COUT/P \rceil \cdot \max(1, m), COUT) \quad (1)$$

cycles: the load of the next window, the compute of $\lceil COUT/P \rceil$ output groups, and the output of COUT results at one

per cycle. Each layer also pays a one-time fill and drain of min(load, compute of the last window) + 3 + g cycles, g being the outputs in the last group of P (16 here), a rule read off board runs of four networks at W = 1 and 4 (g off RTL simulation of a 45-channel network); the first layer, which reads the image, has a measured constant (17 cycles here). We call (1) summed over the windows the count, and the count plus the fill and drain the exact prediction.

Bank of a tap. In (ky, kx, channel) order, tap t = s·N + p, with s the 3×3 position, p the channel index and N the input channels, sits in bank t mod T. If N is a multiple of 4, that is p mod 4: each bank holds N/4 whole channels, and a permutation of the input channels decides which share a bank, changing m in (1) and, through the last window, the fill and drain. We keep the (ky, kx, channel) order, which a hardware line buffer would typically produce; as this host builds the windows, the taps inside a window could also be reordered with the weight columns (Section VII).

When a relabeling can pay. A bank holds at most ⌈K/T⌉ taps, so m ≤ ⌈K/T⌉. If

$$\lceil \mathrm{COUT}/P \rceil \cdot \lceil K/T \rceil \le \max(\lceil K/W \rceil + 1, \mathrm{COUT}), \quad (2)$$

no window is bound by its compute for any input, and no relabeling can change the count; failing (2) is necessary for a gain. Without the rounding, (2) reads COUT ≤ P·T/W, the ridge point of a roofline [12] whose bandwidth roof is the load. In Table I, the 32-output hidden layers (K = 288) need at most 144 cycles of compute, one less than the W = 2 load, so a relabeling can pay there only at W = 4; layers with 45 or 64 outputs can pay at W = 2 and 4, no layer at W = 1. Two lower bounds serve as references: perfect balance, m = ⌈n/T⌉ for n nonzero taps, which no assignment of a window's taps to lockstep banks can beat, and the floor, the count if no window were compute-bound.

TABLE I. WHERE A RELABELING CAN CHANGE THE COUNT, FROM (2). MAX.: ⌈COUT/P⌉·⌈K/T⌉. FLOOR: MAX(⌈K/W⌉ + 1, COUT). NO LAST LAYER (COUT ≤ 4) CAN PAY.

Hidden layers	K / COUT	Max.	Floor, W = 2, 4	Pays, W = 1, 2, 4
A, B-32 ReLU	288 / 32	144	145, 73	no, no, yes
B	576 / 64	576	289, 145	no, yes, yes
maxout, avg.	288 / 64	288	145, 73	no, yes, yes
10 × 45	405 / 45	306	204, 103	no, yes, yes

III. FUNCTION-PRESERVING RELABELING

Folding. Let π map a position p of a hidden map to its original channel. The producer computes its output channels in the order π, with its weight rows, bias values and requantization scales reordered; the consumer's weight column for tap s·N + p is taken from channel π(p). ReLU and requantization act per channel, and the consumer adds the same INT32 products in another order, so the integer output is identical for every input. With L layers we permute the outputs of conv1 to conv(L−2); the last layer, which can never pay, keeps its input. In the maxout [13] and averaging networks a hidden layer computes 2N maps and the host keeps the maximum or mean of maps j and j + N, which move with the same π. For N = 45 the bank of tap s·45 + p is (s + p) mod 4, so every channel's nine taps already fall into all four banks, and a change of class p mod 4 only rotates them.

Fitting. Relabeled windows are original windows with permuted columns, so each layer is fitted alone. The recipe, called Frozen, draws 2048 windows per fitted layer from Set12 01.png (Cal-1; σ = 25, one fixed noise draw, host emulation with the driver's integer arithmetic), minimizes the mean of (1) at W = 4 over them, and searches from the identity with 400 proposals, each swapping two channels of different classes and kept only if the objective falls. One relabeling is fitted per network (7.1–28.8 s on a 4-core x86 server, repeated when the network is retrained) and evaluated at W = 1, 2 and 4. A second layout of the board network, Multi-start, was fitted on Set12 01–03 with two noise draws each (Cal-3) by the same search from the identity and four random starts (2005 objective evaluations per layer against 401); it was chosen for the board after its BSD68 counts had been seen.

IV. EXPERIMENTAL SETUP

Networks and images. Ten trained integer networks of six DnCNN-style architectures [14], for Gaussian noise of σ = 25 (0–255 scale) on 7-bit inputs: A, 7 layers of 32 channels; B, 10 layers of 64 channels behind a 2×2 space-to-depth input; B-32, the same with 32 channels, with ReLU (training seeds s1–s3), maxout over pairs of 64 computed outputs (s1–s3) or the mean of such pairs (s2); and a 45-channel ReLU network with the same input and depth. All but A had quantization-aware training [15]. The board network, B-32 ReLU s1, reaches 29.54 dB on Set12 05.png; 37–55% of its hidden layers' window values are nonzero. Test images: Set12 [14] and BSD68 [16], 80 images with one noise draw each; headline figures exclude the calibration images (79 images for Frozen, 77 for Multi-start).

Board. The W = 4 bitstream for the XC7Z020 (xc7z020clg484-1) closes timing at 100 MHz (worst negative slack +0.130 ns), uses 11,151 LUTs (21%), 71 of 140 BRAM36-equivalent tiles and no DSP slice, and ran at the 50 MHz the board preset gave the logic (a W = 1 run at 100 MHz gave the same counts). One bitstream (core ID 0x4d410303, SHA-256 11641ab… in the run notes) ran all three layouts; an earlier run of the Original used the same build directory. Per layer the driver checks the output against the integer model and reads the cycle counter CYCLES and the counter IN_STALL of cycles spent waiting for the DMA. On Set12 05.png the Original ran once as a canary, then each layout three times in alternation. These board cycles are transcribed from the run notes, each checked by a script against the line it cites; the earlier run is a committed raw report. All other cycle numbers are counts over the windows of the host emulation, which reproduces the reference PSNR of every network on all 80 images.

What was fixed when (GitHub push times, UTC). On 2026-10-17 the W = 4 count model was pushed before the bitstream was built (7e81f6b, 07:52), and the fill-and-drain rule (6dc0e60, 13:15; b37f971, 15:49) was read off board runs of the four unpermuted networks, so the board network's Original layers are in sample. Frozen's per-layer count savings on 05.png (9cf9c79,

18:38) and the two exports (c2ad6ad, 19:25; 45b4f16, 21:00) followed, and the board runs on 2026-10-18. A protocol (c2299e2, 10-18 18:32) then fixed, before this recipe was fitted on any of the other nine networks, the recipe, the evaluation, condition (2) for every layer and four hypotheses (Section V-C); other search methods had already been counted on B and maxout s1 and had shown a W = 4 saving there. The fitted layouts and the evaluation code were pushed in b3461cf (19:29), before the results were written.

V. Results

A. Board Measurement

On Set12 05.png (Table II), Multi-start lowered the board total from 13,289,030 to 12,278,419 cycles (−7.60%) and Frozen to 12,355,814 (−7.02%), from 0.266 s to 0.246 s and 0.247 s of core time at 50 MHz. The repeats gave the same numbers, except that Frozen's third run measured 12 cycles more, all in conv10, where IN_STALL was also 12 (the layer was marked DMA-bound; its values were correct). The 90 layer runs of the repeats had no mismatch, and the notes report the three layouts' denoised images as byte-identical.

In all 63 comparisons (layers, conv2–9 sums, totals and three derived Frozen conv1 + conv10 values of the canary, the repeats and the earlier run), CYCLES − IN_STALL equaled the exact prediction, including the earlier run of the Original (13,289,042 cycles, 12 of them IN_STALL in conv5). Of the 25 that involve the relabeled layouts, 21 are labeled out of sample, but as the repeats are deterministic and conv1 and conv10 see the same windows in every layout, nine are independent and out of sample, all on one image: the Multi-start layers conv2–conv9 and Frozen's conv2–9 sum. The fill-and-drain rule also holds in RTL simulation, which has no DMA. The count alone gives Multi-start's saving as 1,010,573 cycles, 38 fewer than measured: the change of the fill and drain (574 to 536).

Mechanism (counted). The mean nonzero count of the fullest bank fell in every relabeled layer (conv9, Multi-start: 43.04 to 35.10), and more windows reached the load floor of 73 cycles (conv9: 27.8% to 61.2%). Perfect balance would give 11,761,449 cycles (−11.49%) and the floor 11,436,032 (−13.94%); Multi-start removed 66.2% of the perfect-balance saving, Frozen 61.1%.

TABLE II. CORE CYCLES OF B-32 RELU S1 ON SET12 05.PNG AT W = 4. UPRIGHT: BOARD CYCLES TRANSCRIBED FROM THE RUN NOTES, AND SAVINGS FROM THEM. ITALIC: EXACT PREDICTION OR COUNT. M: MULTI-START; F: FROZEN.

Layer	Original	Multi-start	Frozen	Saving, M/F (%)
conv1	524,305	524,305	*524,305*	0.00/*0.00*
conv2	1,504,752	1,378,763	*1,391,312*	8.37/*7.54*
conv3	1,435,005	1,344,509	*1,362,489*	6.31/*5.05*
conv4	1,448,350	1,311,511	*1,311,569*	9.45/*9.44*
conv5	1,562,936	1,371,900	*1,387,589*	12.22/*11.22*
conv6	1,369,221	1,272,788	*1,275,254*	7.04/*6.86*
conv7	1,375,441	1,281,427	*1,291,330*	6.84/*6.12*
conv8	1,289,021	1,207,219	*1,212,745*	6.35/*5.92*
conv9	1,436,476	1,242,474	*1,255,698*	13.51/*12.58*
conv10	1,343,523	1,343,523	*1,343,523*	0.00/*0.00*
conv2–9	11,421,202	10,410,591	10,487,986	8.85/8.17
Total	**13,289,030**	**12,278,419**	**12,355,814**	**7.60/7.02**
Count	*13,288,456*	*12,277,883*	*12,355,274*	*7.60/7.02*
Fill/drain	*574*	*536*	*540*	—

Notes to Table II. Per layer: repeat 1; Frozen was reported on the board only as totals and conv2–9 sums. In every upright cycle cell, board − IN_STALL − exact prediction = 0 (per-layer IN_STALL 0 by the notes' statement that only Frozen's third repeat stalled: 12,355,826 cycles, 12 in conv10). Out of sample: conv2–conv9 of Frozen and Multi-start.

B. Eighty Test Images

At W = 1 and 2 both layouts counted exactly as the original on all 80 images, as (2) requires. At W = 4, Frozen saved 5.13–10.94% per image on the 79 images other than its calibration image (median 7.22%, pooled 7.46%), and Multi-start 5.62–11.19% on the 77 other than its three (median 7.89%, pooled 8.11%); perfect balance would save 7.99–16.62% (pooled 12.02%), the floor 15.01% pooled. Of the 632 fitted image-layers (one fitted layer of one image; 79 × 8), one counted more under Frozen (BSD68 test028 conv3, +1.56%), none of Multi-start's 616. From the smallest saving, 05.png ranks 34th (Frozen) and 32nd (Multi-start) of 80: not a favorable pick. Multi-start counted fewer cycles than Frozen on 74 of the 77 images used by neither fit.

C. Study of Ten Networks

The protocol fixed two checks and two sign predictions (Table IV). H0 (implementation): our code reproduced the earlier Frozen layout in all eight layers, and folding it reproduced the Frozen export byte for byte. H1 (model and code): wherever (2) holds, at W = 1 on all networks and at W = 2 on A and B-32 ReLU s1–s3, the relabeled count equaled the original on all 80 images. H2: the pooled W = 4 saving over the 79 images was positive on all ten networks, 1.09–10.80% (a new prediction for seven; the sign was known for B, maxout s1 and the board network). H3: the pooled W = 2 saving was positive on all six networks where (2) allows it, 3.36–7.37% on five and a negligible 0.022% on the 45-channel network, where 16 of 632 image-layers got worse and conv7 rose by 0.0001%, the only pooled increase of any layer. Only signs were predicted, not effect sizes. No image counted more cycles on any network at any W.

Maxout s1 gained only 3.50% at W = 4; without per-network perfect-balance headroom, not in Table IV, a weak fit cannot be told from a small opportunity. The maxout and averaging networks (K = 288, 64 computed outputs) save almost as much at W = 2 as at W = 4 (averaging: 7.37% and 7.73%): with four output groups, their compute exceeds the W = 2 load of 145 cycles whenever the fullest bank holds more than 36 nonzeros. At W = 4 the calibration image saved more than the median on every network (in sample), and Set12 and BSD68 agreed within 0.8 points. In emulation every folded network reproduced the

original's PNG bytes on Set12 05.png and BSD68 test001.png, with every layer's windows and INT32 accumulators equal up to the column order. The protocol's search routine rejects N = 45, so that network was fitted with the general bank rule (identical layouts on the 69 layers where both apply).

TABLE IV. TEN-NETWORK STUDY: POOLED SAVING OVER THE 79 IMAGES OTHER THAN THE CALIBRATION IMAGE (COUNT, CONV1–CONVL); 0 AT W = 1 ON EVERY NETWORK AND IMAGE. 0‡: (2) HOLDS FOR EVERY LAYER, AND THE SAVING IS EXACTLY 0 ON EVERY IMAGE. WORSE: FITTED IMAGE-LAYERS THAT COUNTED MORE THAN THE ORIGINAL, OF 632 (A: 395).

Network	W = 2 (%)	W = 4 (%)	W = 4, per image (%)	Worse (W = 2/4)
A (7 × 32)	0‡	7.98	6.68–10.59	0 / 0
B (10 × 64)	5.95	9.08	7.31–10.89	2 / 0
B-32 ReLU s1	0‡	7.46	5.13–10.94	0 / 1
B-32 ReLU s2	0‡	10.80	7.39–14.83	0 / 1
B-32 ReLU s3	0‡	9.85	8.58–11.30	0 / 1
B-32 maxout s1	3.36	3.50	2.70–4.61	3 / 3
B-32 maxout s2	6.75	7.04	6.38–8.18	0 / 0
B-32 maxout s3	6.08	6.31	4.90–8.06	0 / 0
B-32 avg. s2	7.37	7.73	6.36–9.45	0 / 0
10 × 45 ReLU	0.022	1.09	0.82–1.36	16 / 0

D. Baselines

Table V compares other layouts of the board network. Over 20 random layouts with eight channels per class the pooled saving ranged from −5.43% to +1.73% (median −2.31%), so savings against the identity understate those against a random order; the one with the best calibration objective saved 1.31% (not a budget-matched search).

Density. A SparTen-style density sort [8] with activation density, dealing the channels by mean nonzero count to the classes in round-robin (0, 1, 2, 3, 0, …) or snake order (0, 1, 2, 3, 3, 2, 1, 0, …), saved 3.59% and 2.56%; assigning each channel, in decreasing count, to the class with the smallest total saved 3.69%, 48% of the Frozen recipe's five-seed mean of 7.62%. The six static density layouts (three per calibration set) captured 21–33% of the perfect-balance saving, the deployable window-level layouts of Table V 62–68.5%. All three Cal-1 density layouts made conv7 slower, the deals also conv3 (snake by 8.41%, and conv6 by 0.17%); every window-level layout lowered every layer's pooled count. A Sense-like sort [6] by each channel's nonzero count in the current image saved 3.74–4.89%, at most 58% of the Frozen recipe fitted on each test image (8.45%); this host could compute such statistics in software, a core with resident weights would need ranking hardware. A density-only heuristic thus reached about half of the window-level saving. This compares procedures; it is not an ablation that isolates co-activation, and the count also depends on correlation among the nine taps of one channel, which no channel relabeling can change.

Search budget. On the 77 images used by neither calibration, Multi-start (8.11%) leads Frozen (7.48%) by 0.63 points; changing one thing at a time, the recipe's five-seed mean on Cal-1 adds 0.15 (the Frozen seed was the lowest of five), five times the budget 0.29, Multi-start's data 0.32, the restarts −0.07 and its seed −0.06 (also the lowest of five). With equal data and budget, plain greedy search (8.24%, five-seed mean) beat Multi-start (8.17%) on four of five seeds.

TABLE V. LAYOUTS OF B-32 RELU S1 AT W = 4 (COUNT). CAL-1: SET12 01, 2048 WINDOWS PER LAYER; CAL-3: SET12 01–03, TWO NOISE DRAWS, 6144 WINDOWS PER LAYER. P.I.: USES THE TEST IMAGE. RECIPE: FROZEN'S. FIVE-SEED ROWS: MEAN POOLED SAVING (RANGE OF THE SEEDS) AND PER-IMAGE RANGE OVER ALL SEEDS; 5×: 2005 EVALUATIONS PER LAYER. SHARE: OF THE PERFECT-BALANCE SAVING. RANDOM: RANGE AND SHARE FROM THE PER-IMAGE MEDIAN.

Layout (79 images; † 77)	Pooled (%)	Per image (%)	Share (%)
Random, median of 20	−2.31	−5.30 to 1.18	−19.2
Density, round-robin, Cal-1	3.59	1.23–6.52	29.9
Density, smallest total, Cal-1	3.69	1.76–5.64	30.7
Sense-like, smallest total (p.i.)	4.89	1.53–9.94	40.7
Frozen (board layout), Cal-1	7.46	5.13–10.94	62.1
Recipe, Cal-1 (7.46–7.78)	7.62	5.13–11.30	63.4
Recipe per test image (p.i.)	8.45	5.77–12.79	70.3
Multi-start, Cal-3† (8.11–8.26)	8.17	5.52–11.55	67.9
Greedy 5×, Cal-3† (8.07–8.36)	8.24	5.39–11.69	68.5

VI. RELATED WORK

Cnvlutin's lanes have our banks' maximum-over-units cost [1]; SCNN reports idle cycles at inter-PE barriers [3], and in SparTen each input-map broadcast imposes an implicit barrier on a cluster's compute units [8]. Balanced pruning [17], [18], top-k activation pruning [4], [5] and BRIM's offline weight reshaping from profiled activation statistics, plus hardware for the residual imbalance [19], change the function; Sense ranks input channels by nonzero count in hardware [6], and Crane moves work between PEs [7]. SparTen's software balancing sorts filters by density offline and reorders the next layer's weights [8], the closest precedent for the mechanism, with a weight objective; Pool and Yu fold a channel permutation for N:M pruning into the preceding layer [11]; Xu et al. and SparHiXcel-v2 order channels on FPGAs for weight sparsity [9], [10]. VARA permutes inputs offline to cluster co-zero activations, applying the order through an index unit [20]. Skewed storage places a matrix in parallel memories so that rows and columns can be accessed without bank conflicts [21].

Channel permutation, its folding, offline activation profiling, greedy search and skewed bank assignment are thus not new; our search (Crossref, arXiv, web; October 2026) found no work that spreads co-active channels over lockstep banks by an exact, folded relabeling. The contribution is empirical: the share of the bank-imbalance cost one such relabeling removes on unseen images, and exact agreement of predicted and measured cycles.

VII. LIMITATIONS

• **Tap-level layouts.** As this host builds the windows, any static order of the K taps, applied to the windows and the

weight columns, is also exact; this larger space includes layouts that spread each channel's nine taps over all four banks, as in the 45-channel layers. It was not evaluated systematically; in a preliminary count (not yet in the repository) one data-free skew of this kind saved more on the board network than both fitted layouts: the relabeling is not shown to be the best static layout, only one that keeps the window order.

• **Board evidence.** One network and one image were measured; the nine out-of-sample checks differ from the in-sample ones only in the layout, and the fill-and-drain constants come from runs of the same image. Table II is transcribed from run notes that give neither Frozen's per-layer cycles nor image digests; the raw reports are not yet committed.

• **End-to-end effect.** Only core cycles are reported. The JTAG test harness dominates: the earlier run of the Original spent 315.0 s in its ten layers' debugger sessions against 0.266 s of core time, so Multi-start's 20.2 ms are 0.006%. Host im2col and requantization on the ARM cores, energy, and banks decoupled by FIFOs, which could beat the per-window perfect-balance bound, were not measured.

• **Statistics and selection.** Only $\sigma = 25$ with one noise draw per image was evaluated, though sparsity changes with the noise level; $W = 2$ results are counts. The spread over the choice of calibration image was not measured, and pooled savings have no confidence interval. BSD68 was part of the networks' development, and Multi-start was chosen after its BSD68 counts were seen.

• **Stall counter.** This bitstream's IN_STALL misses DMA starvation at window boundaries (fixed in later RTL); the exact match excludes hidden stalls that differ between layouts, and the rule's hold in RTL simulation suggests the constants contain none.

VIII. Conclusion

On a zero-skipping core whose four banks run in lockstep and whose windows keep the (ky, kx, channel) order, the channel order sets the bank balance, and the layer shapes tell where it can matter. Two relabelings fixed on calibration images cut the measured core cycles of one image by 7.02% and 7.60% with identical output, as predicted to the cycle after reported stalls, and the pooled count fell on all ten networks of a study whose protocol was pushed before the fits. Tap-level layouts, more board images, other noise levels and end-to-end time remain open. Scripts, layouts, per-image counts and a map from every number to its source are in bankperm/ of https://github.com/qewr1234/FPGA.

Acknowledgment

(Acknowledgment)

References

[1] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. Enright Jerger, and A. Moshovos, "Cnvlutin: Ineffectual-neuron-free deep neural network computing," in *Proc. ACM/IEEE 43rd Annu. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 1–13, doi: 10.1109/ISCA.2016.11.

[2] A. Aimar et al., "NullHop: A flexible convolutional neural network accelerator based on sparse representations of feature maps," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 3, pp. 644–656, 2019, doi: 10.1109/TNNLS.2018.2852335.

[3] A. Parashar et al., "SCNN: An accelerator for compressed-sparse convolutional neural networks," in *Proc. 44th Annu. Int. Symp. Comput. Archit. (ISCA)*, 2017, pp. 27–40, doi: 10.1145/3079856.3080254.

[4] Z.-G. Liu, P. N. Whatmough, Y. Zhu, and M. Mattina, "S2TA: Exploiting structured sparsity for energy-efficient mobile CNN acceleration," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2022, pp. 573–586, doi: 10.1109/HPCA53966.2022.00049.

[5] E. Yan, D. Xu, and J. Liu, "Bank-balanced-sparse activation feature maps for neural network models," U.S. Patent 12 236 341 B2, Feb. 25, 2025.

[6] W. Sun, D. Liu, Z. Zou, W. Sun, S. Chen, and Y. Kang, "Sense: Model-hardware codesign for accelerating sparse CNNs on systolic arrays," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 4, pp. 470–483, 2023, doi: 10.1109/TVLSI.2023.3241933.

[7] Y. Guan et al., "Crane: Mitigating accelerator under-utilization caused by sparsity irregularities in CNNs," *IEEE Trans. Comput.*, vol. 69, no. 7, pp. 931–943, 2020, doi: 10.1109/TC.2020.2981080.

[8] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2019, pp. 151–165, doi: 10.1145/3352460.3358291.

[9] J. Xu, H. Pu, and D. Wang, "Sparse convolution FPGA accelerator based on multi-bank hash selection," *Micromachines*, vol. 16, no. 1, Art. no. 22, 2025, doi: 10.3390/mi16010022.

[10] A. Zarei and S. Vakili, "A flexible sparsity-aware FPGA accelerator with column-wise compression for efficient CNN inference," arXiv:2607.19248, 2026.

[11] J. Pool and C. Yu, "Channel permutations for N:M sparsity," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34, 2021, pp. 13316–13327.

[12] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009, doi: 10.1145/1498765.1498785.

[13] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio, "Maxout networks," in *Proc. 30th Int. Conf. Mach. Learn. (ICML)*, PMLR vol. 28, 2013, pp. 1319–1327.

[14] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, 2017, doi: 10.1109/TIP.2017.2662206.

[15] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713, doi: 10.1109/CVPR.2018.00286.

[16] S. Roth and M. J. Black, "Fields of experts," *Int. J. Comput. Vis.*, vol. 82, no. 2, pp. 205–229, 2009, doi: 10.1007/s11263-008-0197-6.

[17] S. Han et al., "ESE: Efficient speech recognition engine with sparse LSTM on FPGA," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, 2017, pp. 75–84, doi: 10.1145/3020078.3021745.

[18] S. Cao et al., "Efficient and effective sparse LSTM on FPGA with bank-balanced sparsity," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, 2019, pp. 63–72, doi: 10.1145/3289602.3293898.

[19] V. Manjunath, R. Yin, D. Lee, A. Ghosh, and P. Panda, "BRIM: Workload-balanced dual-sided bit-serial sparse inference accelerator," arXiv:2607.19431, 2026.

[20] P. Dang, Y. He, and H. Li, "VARA: A voltage-aware ReRAM-based accelerator for energy-efficient computing," arXiv:2609.00421, 2026.

[21] P. Budnik and D. J. Kuck, "The organization and use of parallel memories," *IEEE Trans. Comput.*, vol. C-20, no. 12, pp. 1566–1569, 1971, doi: 10.1109/T-C.1971.223171.